

SML Assignment 4

Q1

Methodology

Data Preparation and Dimensionality Reduction

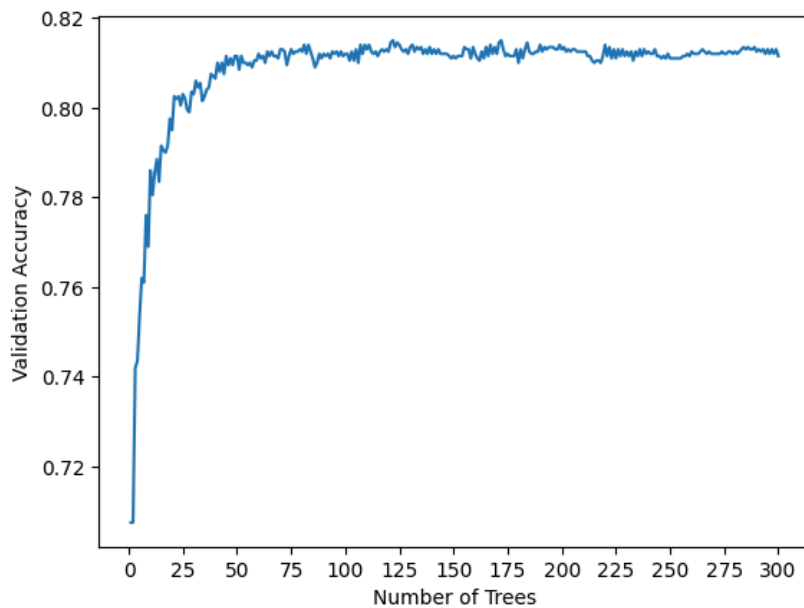
- **Data Preprocessing:** The MNIST dataset is loaded, and classes 4 and 9 are filtered and relabeled as -1 and 1, respectively.
- **Normalization:** Images are flattened and normalized by dividing pixel values by 255.0 to maintain a range of [0, 1].
- **Validation Split:** A portion of the training data (1,000 samples per class) is set aside for validation, ensuring these samples are not used during the training of the weak learners or the PCA calculation.
- **Dimensionality Reduction:** Principal Component Analysis (PCA) is applied to reduce the feature space to 5 dimensions using only the training set to compute the projection matrix.

AdaBoosting Algorithm

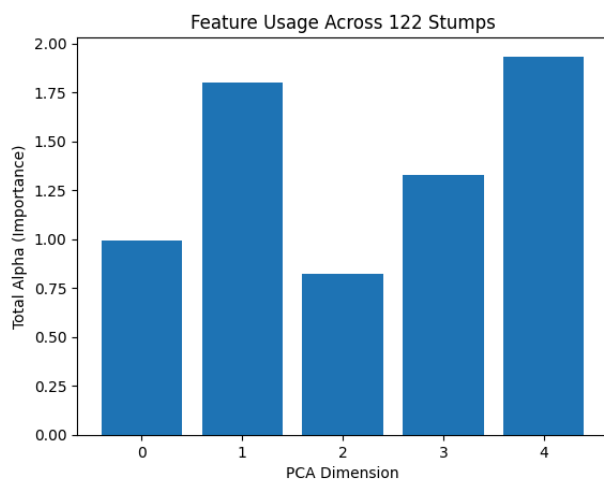
- **AdaBoost Training:** An ensemble of 300 decision stumps is grown iteratively.
 - **Weak Learner:** For each stump, 1,000 random candidate thresholds (midpoints between sorted values) are evaluated across all 5 dimensions to find the split minimizing the weighted misclassification error.
 - **Weight Updates:** Sample weights are updated after each iteration to give more importance to misclassified samples.
 - **Alpha Calculation:** Each stump is assigned a weight based on its error, determining its influence in the final cumulative model.
- **Model Selection:** The cumulative validation accuracy is tracked at every step to identify the best iteration where the ensemble performance peaks. (Following the principle of Occam's Razor, the model at the first occurrence of the maximum validation accuracy is reported. This ensures the chosen ensemble is the simplest version that achieves peak performance, reducing the risk of overfitting the noise in the training data.)

Results

Metric	Value
Best Iteration	122
Validation Accuracy	0.8150
Test Set Accuracy	0.8227



This plot displays the cumulative performance of the ensemble. Typically, the accuracy rises sharply in the early iterations as the boosting algorithm corrects major errors, then slightly fluctuates as it tries to fix hard outliers, and then saturates.



The plot shows the importance of each feature by aggregating the voting weights (α) across 122 stumps, revealing that PCA Dimension 4 and Dimension 1 are the primary drivers for classification in the AdaBoost ensemble.

Q2

Methodology

Data Preparation and Dimensionality Reduction

- **Dataset:** The MNIST dataset was filtered to include only classes 4 and 9.
- **PCA:** To handle the high dimensionality of the images, PCA was applied to reduce the feature space to the top 5 principal components.
- **Normalization:** Input features were normalized by dividing pixel values by 255.0 to ensure numerical stability during the SSR calculations.

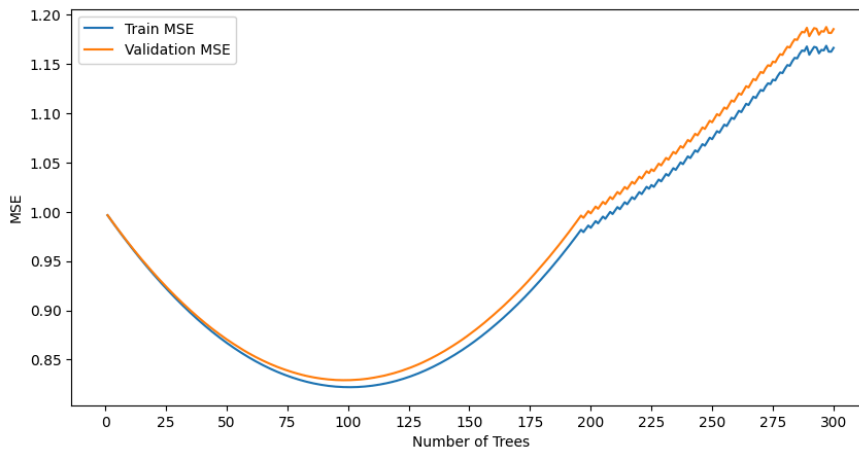
Gradient Boosting Algorithm

- **Weak Learners:** We used regression stumps that minimize the SSR.
- **Optimization:** For each stump, we sampled up to 1000 random thresholds per PCA dimension to efficiently find the split that minimizes SSR.
- **Additive Updates:** The model was built iteratively. At each step, a stump was trained on the sign of the current residuals, and its prediction was added to the ensemble scaled by a learning rate.

Results

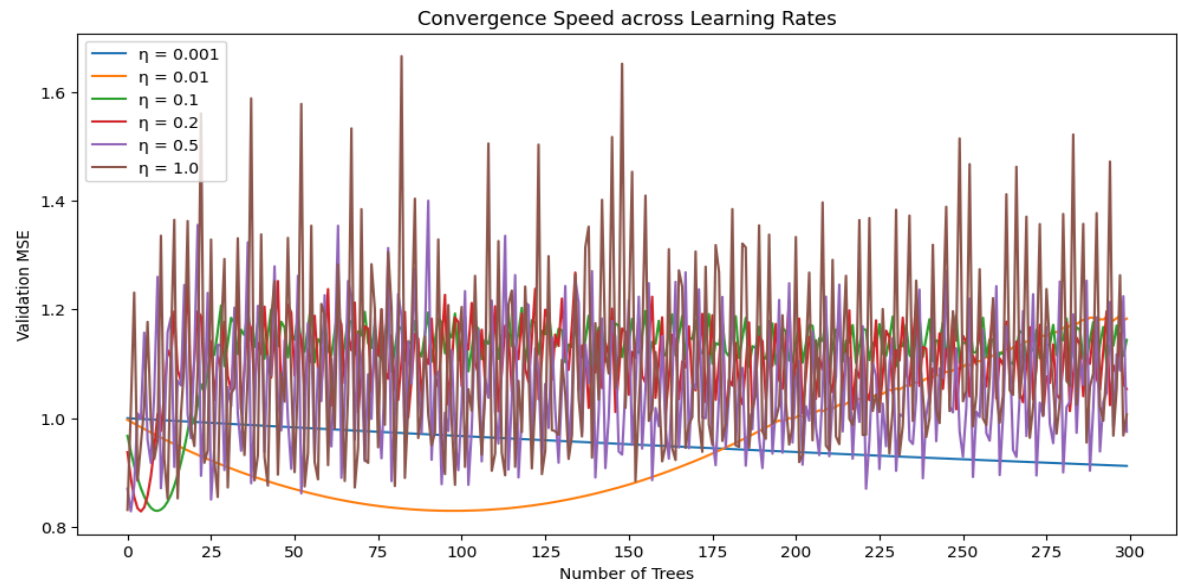
For the primary experiment with a learning rate of **0.01** over **300 iterations**, the following results were obtained:

- **Optimal Number of Trees:** The model reached its minimum validation error at **Iteration 98**.
- **Performance Metrics:**
 - **Best Validation MSE:** 0.8291
 - **Final Test MSE:** 0.8355



For the primary experiment with $\eta=0.01$, both the **Train MSE** and **Validation MSE** exhibit a distinct **U-shaped trajectory**. The error initially decreases smoothly, reaching a global minimum at **Iteration 98**, before beginning to rise steadily as the number of trees increases. This clear upward trend in the validation curve after the 98th iteration indicates that the model starts to **overfit** the training noise. The close tracking between the training and validation curves throughout this process suggests that the model maintains high generalization stability, even while transitioning from underfitting to overfitting.

Learning Rate Comparison



Learning Rate (η)	Optimal Trees (Best T)	Min Val MSE	Final Test MSE
0.500	2	0.82819	0.83432
0.100	10	0.82897	0.83496
0.010	99	0.82914	0.83539
0.200	5	0.82797	0.83540
1.000	1	0.83080	0.83680
0.001	300	0.91182	0.91413

High Learning Rates ($\eta \geq 0.2$): These values demonstrate extreme instability when using absolute loss gradients, characterized by high-frequency oscillations in the Validation MSE. However for $\eta=1.0$, the model reaches its minimum error at the very first tree before performance degrades.

Low Learning Rates ($\eta \leq 0.01$): These curves are significantly smoother and exhibit clear convergence towards a minimum. Specifically, at $\eta=0.01$, the model reaches its optimal performance at **iteration 98**. However, $\eta=0.001$ remains underfitted after 300 iterations with a high MSE of **0.91182**, as the incremental steps are too small to capture the data's complexity within the given limit.

Optimal Performance: $\eta=0.500$ achieved the lowest overall **Final Test MSE of 0.83432**, reaching its optimal state after only 2 trees.

Conclusion

The Gradient Boosting approach effectively distinguishes between digits 4 and 9 by iteratively correcting the ensemble's prediction direction. While **AdaBoost** optimizes classification by reweighting misclassified samples, this **Gradient Boosting** implementation utilizes the negative gradient of the **absolute loss**.

Q3

Methodology

Data Preparation and Sampling

- **Dataset A Generation:** Two classes were sampled from multivariate normal distributions: Class -1 with $\mu=[-3,-3]^T$ and Class 1 with $\mu=[3,3]^T$, both sharing a covariance matrix $\Sigma=I$.
- **Dataset B Generation:** Classes followed the same means as Dataset A but utilized a much higher covariance $\Sigma=3I$, increasing data spread and overlap potential.
- **Pre-processing:** Each dataset consisted of 400 total samples (200 per class), which were shuffled randomly and partitioned using a 70-30 train-test split.

Rosenblatt Perceptron Implementation

- **Algorithm:** A Perceptron was implemented (from scratch) using the weight update rule $w \leftarrow w + \eta y_i x_i$ and bias update $b \leftarrow b + \eta y_i$.
- **Initialization:** All weights and biases were initialized to zero to ensure a neutral starting decision boundary.
- **Hyperparameters:** The learning rate was set to $\eta=0.01$ with a maximum limit of 300 epochs.
- **Early Stopping:** The training loop incorporated a convergence check that halted execution if a full epoch passed with no misclassifications.

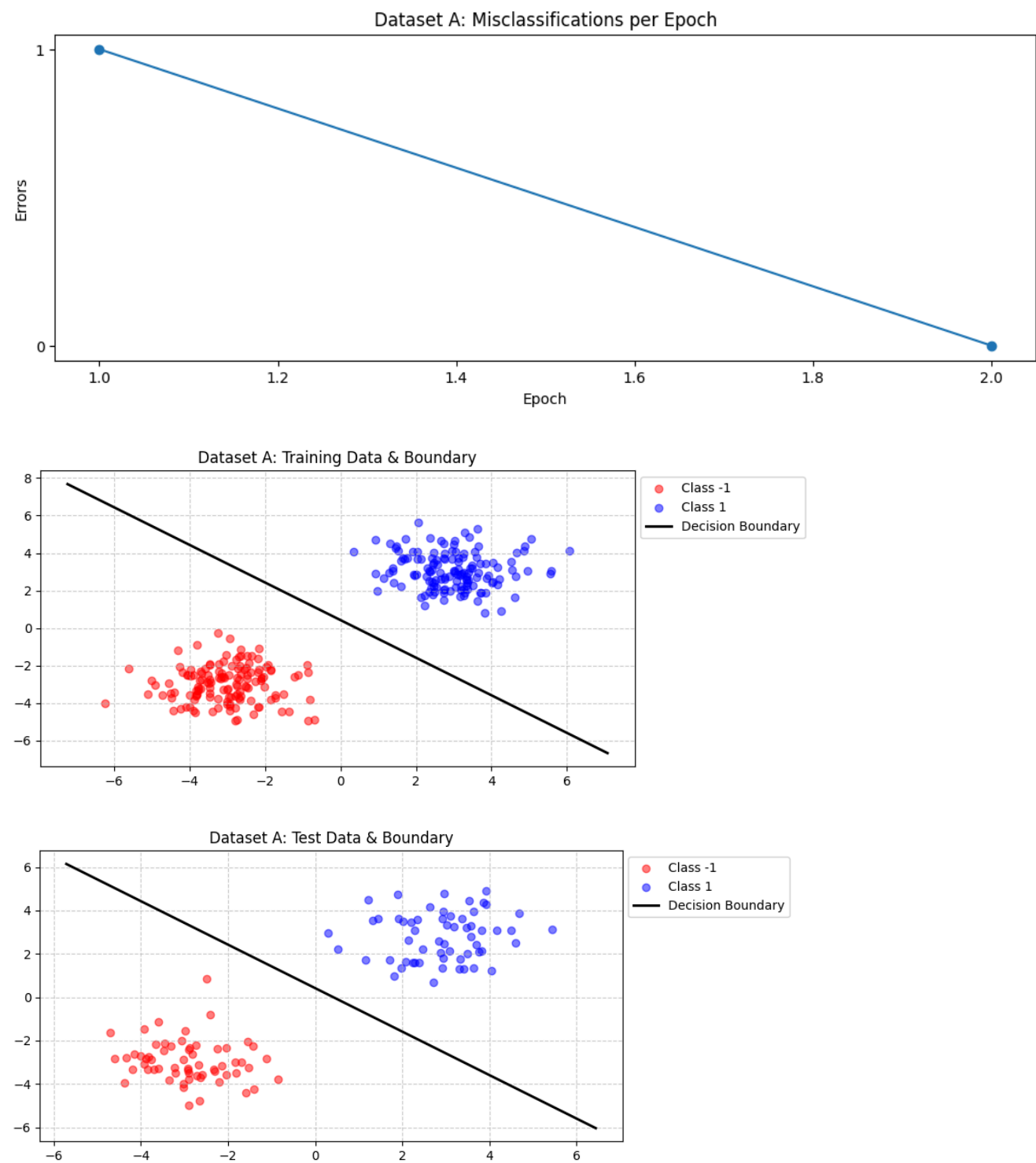
Parameter after training	Dataset A	Dataset B
Weight	[0.0237588, 0.02371654]	[0.03791832, 0.04546908]
Bias	-0.01	-0.019999999999999997

Results

Metric	Dataset A	Dataset B
Convergence Epoch	2	No convergence
Test Accuracy	100.00%	99.17%

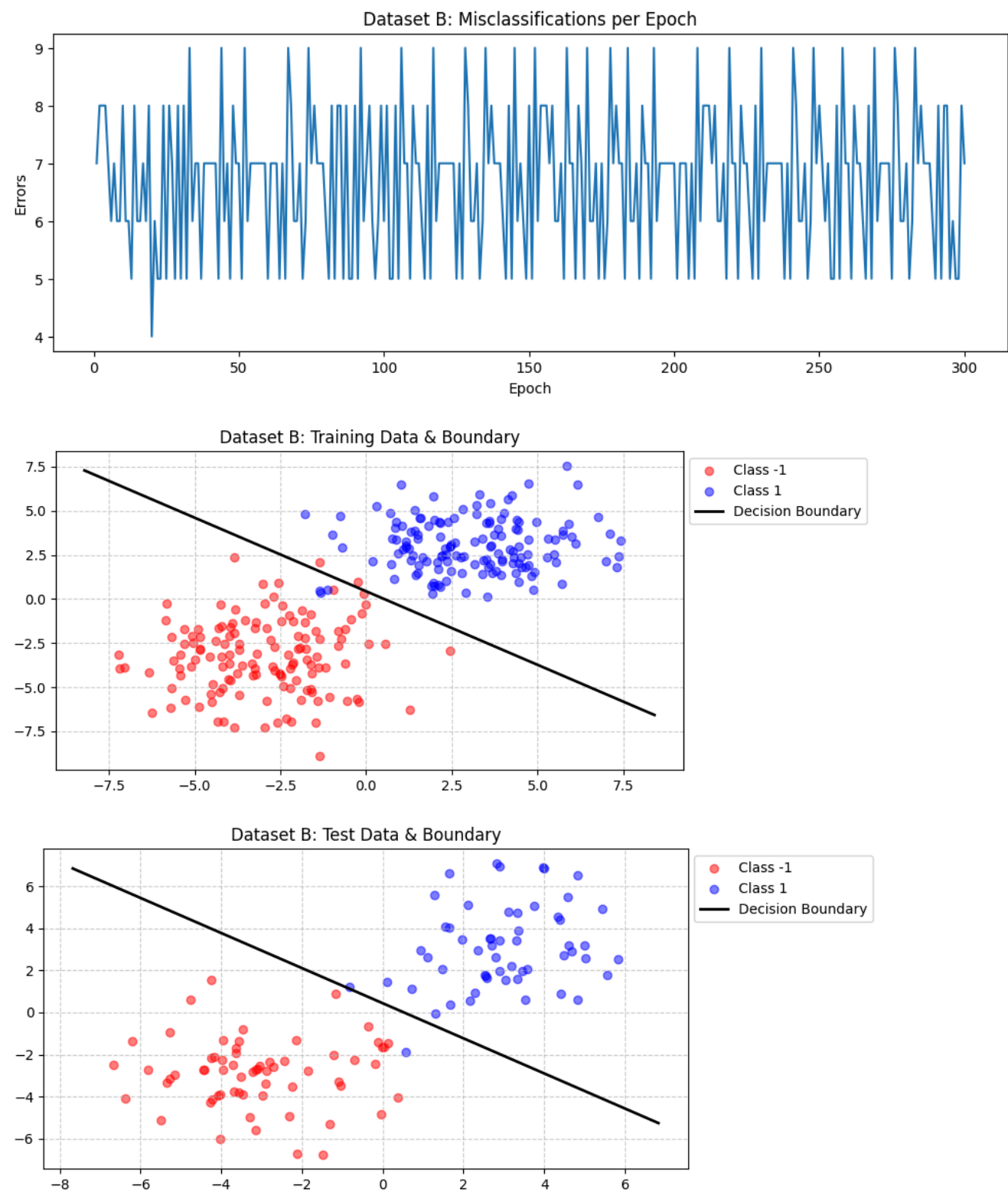
The final test accuracy of Dataset B exhibits minor fluctuations across different runs. Since the model never reaches a global minimum, the final boundary is highly sensitive to the specific sequence of misclassified samples processed just before the 300-epoch limit. Hence, a random seed of 42 was used for reproducibility of the results.

Dataset A: Performance and Boundary



The misclassification plot for Dataset A shows a rapid, monotonic decline, reaching zero errors by the second epoch. The decision boundary clearly separates the two clusters in both the training and test sets with no overlap, confirming perfect linear separability.

Dataset B: Performance and Boundary



The error plot for Dataset B exhibits high volatility and persistent jitter. The errors fluctuate and never reach zero, indicating that the algorithm is trapped in an oscillatory state. The decision boundary plots reveal points from opposite classes encroaching upon the boundary region, illustrating the impact of increased covariance.

Trend Analysis and Justification

- **Linear Separability and Convergence:** Dataset A converged almost instantly because the low covariance ($\Sigma=I$) created a wide margin between the two class means. Perceptron Convergence Theorem guarantees a solution in finite steps for linearly separable data.
- **Linear Separability Validation:** To confirm the observed behaviors, the functional margin condition was evaluated for the final learned hyperplanes.
Dataset A yielded a positive minimum margin (0.0601), verifying its status as a linearly separable problem and justifying the rapid convergence in 2 epochs.
Dataset B yielded a negative minimum margin (-0.0549), verifying that no linear hyperplane can perfectly separate the classes due to the 3I covariance overlap. This confirms that the observed oscillations are a result of the algorithm's inability to satisfy the separability condition.
- **Impact of High Covariance:** In Dataset B, the 3I covariance increased the radius (R) of the data points and decreased the margin (γ). This led to class overlap near the origin, making the dataset linearly non-separable.
- **Oscillation Phenomenon:** The sawtooth pattern in the Dataset B error plot occurs because the Perceptron update rule is forced to continuously shift the hyperplane to account for misclassified points in the overlapping region. Each update that fixes one noise point often causes a previously correctly classified point on the other side to become misclassified, preventing the error from ever reaching zero.

Conclusion

Despite failing to converge, the model achieved a very high test accuracy, suggesting that while a perfect linear separator does not exist, the learned hyperplane successfully captures the underlying distribution of the data.